



Now starting from the beginning give 50 extremely detailed github copilot prompts that fully build the MVP of the WeUP application. Starting from the beginning to the end.

Here are **50 detailed, step-by-step GitHub Copilot prompts** (starting from absolute scratch, in order) that fully build the MVP for the WeUP application, with stack coverage: **Vue.js (Quasar), Python, C#, PostgreSQL, Weviate, Three.js, Mapbox 3D, Unreal Engine, Stripe**. These prompts incorporate all key flows and requirements extracted from your technical and product documents.

1–5: Foundational Backend and API Setup (.NET 8 Microservices, PostgreSQL, Identity/Invite)

1.

```
// Scaffold a .NET 8 microservice for the WeUP Identity/Access Service with JWT login, JW
```

2.

```
// Create invite-only user model in the Identity service: invite code generation, expirat
```

3.

```
// Implement a .NET API endpoint for completing onboarding, syncing new user info to the
```

4.

```
// Scaffold the API Gateway (Ocelot or Azure API Gateway) configuration for routing, rate
```

5.

```
// Using Entity Framework, define full PostgreSQL schemas for: users (tiers, profile), ev
```

6–10: Event, Ticket, and Shop Microservices (C#, PostgreSQL)

6.

```
// Build EventService microservice with endpoints to create, update, list, search, and re
```

7.

```
// Add TicketingService with digital ticket creation, unique QR code generator (QrCoder),
```

8.

```
// Implement real-time ticket scanning/validation endpoint: endpoint receives QR, checks
```

9.

```
// Scaffold ShopService (micro-shop) endpoints for: create shop (per event/booth), add/ec
```

10.

```
// Implement OrdersService to coordinate shop orders, checkout, inventory deduction, paym
```

11–15: Real-Time, Messaging, Notification, and Mapping (SignalR, Mapbox/Three.js)

11.

```
// Set up SignalR-based RealTimeService microservice with hubs for map position updates,
```

12.

```
// In Vue (Quasar), create the onboarding/signup/login page flow with invite code input,
```

13.

```
// Build global notification and chat messaging bus in C# using SignalR; on frontend, cor
```

14.

```
// Create a Typescript wrapper/component for displaying geolocated 3D event maps using Ma
```

15.

```
// Implement BLE beacon integration and GPS fallback in Vue SPA using Capacitor/geolocati
```

16–20: Frontend SPA (Vue.js + Quasar, Routing, UI Flows)

16.

```
// Scaffold main Vue SPA using Quasar; add routes for /home, /login, /discover, /events/:
```

17.

```
// Create a discover feed page in Vue that queries events API for trending/upcoming event
```

18.

```
// Build detailed event page with summary, schedule, performers, RSVP/purchase workflow,
```

19.

```
// Implement micro-shop UI in Vue: shop list, product detail modal, live inventory update
```

20.

```
// Develop full-featured user profile page: view/edit bio and photo, tier badge, event hi
```

21–25: Payment, Inventory, and Order Workflow (Stripe, Real-Time Sync)

21.

```
// Integrate Stripe checkout in backend: API endpoint to create CheckoutSession, link to
```

22.

```
// Build Stripe Elements integration for Vue frontend: pass session ID, redirect to Strip
```

23.

```
// Implement robust inventory locking in ShopService using Redis distributed locks or row
```

24.

```
// On shop product page, subscribe to real-time inventory updates from SignalR and instar
```

25.

```
// Build refund/cancellation workflow (admin or user), REST endpoint to request refund, \
```

26–30: 3D/AR Experience (Three.js, Mapbox, Unreal, Indoor Nav)

26.

```
// Integrate Mapbox 3D (GL JS) and Three.js for the venue map; load venues as geo-referenced
```

27.

```
// Add friend/group real-time sharing toggle: user can opt in to share/show location to s
```

28.

```
// Embed a minimized Unreal Engine 3D scene for detailed indoor navigation as an iframe/c
```

29.

```
// Add AR navigation UI (AR.js/Cordova/Capacitor plugin for web): overlay arrows/directio
```

30.

```
// Develop a simple heatmap overlay for the 3D map showing crowd density, using real-time
```

31–35: Social, Gamification, and Ranking

31.

```
// Implement social graph endpoints in .NET: friend request, accept, block, list friends,
```

32.

```
// Scaffold friend-invite tree: store, visualize, and manage who invited whom, tier path,
```

33.

```
// Implement gamification microservice: award points for onboarding, inviting, event atte
```

34.

```
// Show points, achievements, and leaderboards on the profile page; show event/venue-spec
```

35.

```
// Add daily/weekly challenges and progress UI, with logic for tracking, point awarding,
```

36–40: Advanced AI & Recommendation (Python, Weviate, FastAPI)

36.

```
# Scaffold FastAPI Recommendation microservice (Python) that integrates with Weviate for
```

37.

```
# Build a script to sync transactional/user/event/shop/connection data from PostgreSQL to
```

38.

```
# Implement collaborative filtering and content-based hybrid recommendation in the FastAPI
```

39.

```
// Frontend: consume recommendations API to show “suggested events” and “recommended for
```

40.

```
# Build basic event FAQ bot using OpenAI API/LLM with context retrieval on event metadata
```

41–45: Admin/Analytics/Monitor/Compliance

41.

```
// Scaffold admin dashboard APIs (in .NET): list/search/filter events, orders, users, shop
```

42.

```
// Admin dashboard SPA (Vue + Quasar): analytics charts, GMV, sales/ticket stats, heatmap
```

43.

```
// Add logs and structured audit trail to all backend services; log order/purchase events
```

44.

```
# Automated backup/restore scripts for PostgreSQL, media (S3), and Weviate—schedule daily
```

45.

```
# Set up Prometheus (metrics), Grafana (dashboards), Loki/ELK for logs across all microse
```

46–50: Deployment, CI/CD, Security, and Mobile

46.

```
# Compose Kubernetes manifests to deploy all microservices, frontend SPA (as static files)
```

47.

```
# GitHub Actions pipeline: build/test/deliver each service/container, run database migrat
```

48.

```
// Add 2FA (TOTP, FIDO2, or SMS) for organizer/admin users in identity service, including
```

49.

```
# .NET, Node, Python container images: run security scanning, check for known vulnerabili
```

50.

```
// In Vue Quasar, add PWA capabilities: install prompt, offline ticket display, backgroun
```

These prompts—when used in Copilot—will guide you or your engineering team from empty repo to a working MVP for WeUP, fully covering every foundational, real-time, social, AI, and immersive component in technical and operational detail.

✱

1. WeUP-Event-Platform-Technical-Requirements-Documents-TRD.pdf
2. WeUP-Revolutionizing-Event-Engagement-and-Ticketing.pdf
3. WeUP-2025-2028-Innovation-Roadmap.pdf